1.1 concepts of NoSQL. Advantages and features.
    1.1.1 MongoDB Datatypes (String, Integer, Boolean, Double, Arrays, Objects)
    1.1.2 Database creation and dropping database
1.2 create and Drop collections
1.3 CRUD operations (Insert, update, delete, find, Query and Projection operators)
1.4 Operators (Projection, update, limit (), sort ()) and Aggregation commands

1.1 What is NoSQL?

NoSQL is a type of database management system (DBMS) that is designed to handle and store large volumes of unstructured and semi-structured data. Unlike traditional relational databases that use tables with pre-defined schemas to store data, NoSQL databases use flexible data models that can adapt to changes in data structures and are capable of scaling horizontally to handle growing amounts of data.

The term NoSQL originally referred to "non-SQL" or "non-relational" databases, but the term has since evolved to mean "not only SQL," as NoSQL databases have expanded to include a wide range of different database architectures and data models.

NoSQL databases are generally classified into four main categories:

1. **Document databases:** These databases store data as semi-structured documents, such as JSON or XML, and can be queried using document-oriented query languages.
2. **Key-value stores:** These databases store data as key-value pairs, and are optimized for simple and fast read/write operations.
3. **Column-family stores:** These databases store data as column families, which are sets of columns that are treated as a single entity. They are optimized for fast and efficient querying of large amounts of data.
4. **Graph databases:** These databases store data as nodes and edges, and are designed to handle complex relationships between data.

Features of NoSQL

1. **Non-relational:**
    - NoSQL databases never follow the relational model

Esha Patel

- Never provide tables with flat fixed-column records Work with self-contained aggregates or BLOBs
- Doesn't require object-relational mapping and data normalization
- No complex features like query languages, query planners, referential integrity joins, ACIDs

2. **Schema-free:**
   - NoSQL databases are either schema-free or have relaxed schemas
   - Do not require any sort of definition of the schema of the data
   - Offers heterogeneous structures of data in the same domain
3. **Simple API:**
   - Offers easy to use interfaces for storage and querying data provided
   - APIs allow low-level data manipulation & selection methods
   - Text-based protocols mostly used with HTTP REST with JSON
   - Mostly used no standard based NoSQL query language
   - Web-enabled databases running as internet-facing services
4. **Distributed:**
   - Multiple NoSQL databases can be executed in a distributed fashion
   - Offers auto-scaling and fail-over capabilities
   - Often ACID concept can be sacrificed for scalability and throughput
   - Mostly no synchronous replication between distributed nodes Asynchronous
   - Multi-Master Replication, peer-to-peer, HDFS Replication
   - only providing eventual consistency
   - Shared Nothing Architecture. This enables less coordination and higher distribution.
   - NoSQL is Shared Nothing.

Advantages of NoSQL

- supports query language.
- provides fast performance.
- provides horizontal scalability.
- Can be used as Primary or Analytic Data Source
- Big Data Capability

Esha Patel

- No Single Point of Failure
- Easy Replication
- No Need for Separate Caching Layer.
- Can handle structured, semi-structured, and unstructured data with equal effect
- Object-oriented programming which is easy to use and flexible
- NoSQL databases don't need a dedicated high-performance server
- Support Key Developer Languages and Platforms
- Simple to implement than using RDBMS
- It can serve as the primary data source for online applications.
- Handles big data which manages data velocity, variety, volume, and
- Excels at distributed database and multi-data centre operations
- Eliminates the need for a specific caching layer to store data
- Offers a flexible schema design which can easily be altered without downtime or service disruption

Dis-Advantages of NoSQL

- No standardization rules
- Limited query capabilities
- RDBMS databases and tools are comparatively mature
- It does not offer any traditional database capabilities, like consistency when multiple transactions are performed simultaneously.
- When the volume of data increases it is difficult to maintain unique values as keys become difficult
- Doesn't work as well with relational data
- The learning curve is stiff for new developers
- Open source options so not so popular for enterprises

Introduction of MongoDb

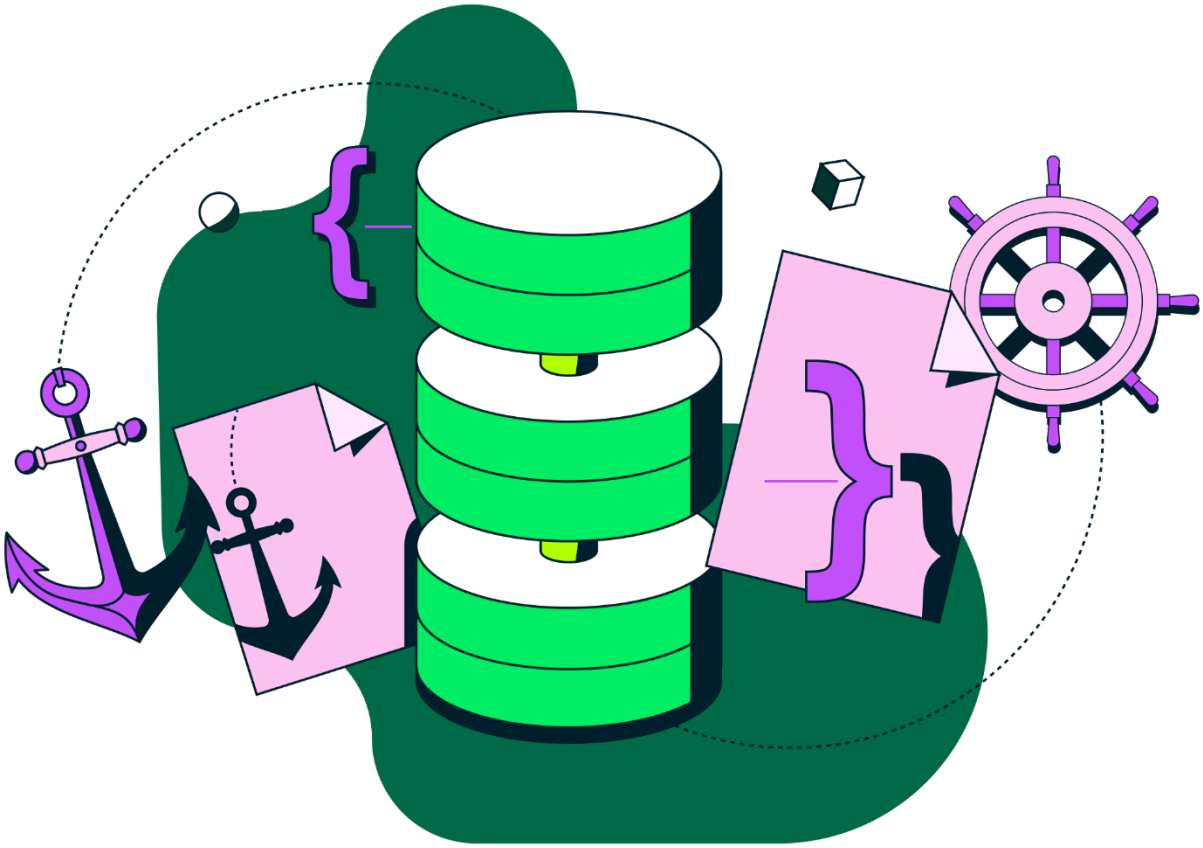
What is MongoDB?

Esha Patel

MongoDB is a document database designed for ease of application development and scaling. MongoDB is an open-source document database and leading NoSQL database. MongoDB is written in C++. MongoDB is a document database. It stores data in type of JSON format called BSON (Binary Javascript Object Notation).

Run MongoDB with

- MongoDB Atlas fully managed in the cloud,
- the source available and free-to-use MongoDB Community, or
- the MongoDB Enterprise Advanced subscription.



1.1.1 MongoDB Datatypes (String, Integer, Boolean, Double, Arrays, Objects)

MongoDB supports many datatypes. Some of them are –

- **String** − This is the most commonly used datatype to store the data. String in MongoDB must be UTF-8 valid.
- **Integer** − This type is used to store a numerical value. Integer can be 32 bit or 64 bit depending upon your server.
- **Boolean** − This type is used to store a boolean (true/ false) value.
- **Double** − This type is used to store floating point values.
- **Min/ Max keys** − This type is used to compare a value against the lowest and highest BSON elements.
- **Arrays** − This type is used to store arrays or list or multiple values into one key.
- **Timestamp** − timestamp. This can be handy for recording when a document has been modified or added.
- **Object** − This datatype is used for embedded documents.
- **Null** − This type is used to store a Null value.
- **Symbol** − This datatype is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.
- **Date** − This datatype is used to store the current date or time in UNIX time format. You can specify your own date time by creating object of Date and passing day, month, year into it.
- **Object ID** − This datatype is used to store the document's ID.
- **Binary data** − This datatype is used to store binary data.
- **Code** − This datatype is used to store JavaScript code into the document.
- **Regular expression** − This datatype is used to store regular expression.

MongoDB is a cross-platform, document oriented database that provides, high performance, high availability, and easy scalability. MongoDB works on concept of collection and document.

Database

Esha Patel

Database is a physical container for collections. Each database gets its own set of files on the file system. A single MongoDB server typically has multiple databases.

Collection

Collection is a group of MongoDB documents. It is the equivalent of an RDBMS table. A collection exists within a single database. Collections do not enforce a schema. Documents within a collection can have different fields. Typically, all documents in a collection are of similar or related purpose.

Document

A document is a set of key-value pairs. Documents have dynamic schema. Dynamic schema means that documents in the same collection do not need to have the same set of fields or structure, and common fields in a collection's documents may hold different types of data.

The following table shows the relationship of RDBMS terminology with MongoDB.

RDBMS	MongoDB
Database	Database
Table	Collection
Tuple/Row	Document
column	Field
Table Join	Embedded Documents
Primary Key	Primary Key (Default key _id provided by MongoDB itself)
Database Server and Client	
mysqld/Oracle	mongod
mysql/sqlplus	mongo

Esha Patel

Sample Document

Following example shows the document structure of a blog site, which is simply a comma separated key value pair.

```
{
  _id: ObjectId(7df78ad8902c)
  title: 'MongoDB Overview',
  description: 'MongoDB is no sql database',
  by: 'tutorials point',
  url: 'http://www.tutorialspoint.com',
  tags: ['mongodb', 'database', 'NoSQL'],
  likes: 100,
  comments: [
    {
      user:'user1',
      message: 'My first comment',
      dateCreated: new Date(2011,1,20,2,15),
      like: 0
    },
    {
      user:'user2',
      message: 'My second comments',
      dateCreated: new Date(2011,1,25,7,45),
      like: 5
    }
  ]
}
```

_id is a 12 bytes hexadecimal number which assures the uniqueness of every document. You can provide _id while inserting the document. If you don't provide then MongoDB provides a unique id for every document. These 12 bytes first 4 bytes for the current timestamp, next 3 bytes for machine id, next 2 bytes for process id of MongoDB server and remaining 3 bytes are simple incremental VALUE.

Any relational database has a typical schema design that shows number of tables and the relationship between these tables. While in MongoDB, there is no concept of relationship.

Advantages of MongoDB over RDBMS

Esha Patel

- **Schema less** – MongoDB is a document database in which one collection holds different documents. Number of fields, content and size of the document can differ from one document to another.
- Structure of a single object is clear.
- No complex joins.
- Deep query-ability. MongoDB supports dynamic queries on documents using a document-based query language that's nearly as powerful as SQL.
- Tuning.
- **Ease of scale-out** – MongoDB is easy to scale.
- Conversion/mapping of application objects to database objects not needed.
- Uses internal memory for storing the (windowed) working set, enabling faster access of data.

Why Use MongoDB?

- **Document Oriented Storage** – Data is stored in the form of JSON style documents.
- Index on any attribute
- Replication and high availability
- Auto-Sharding
- Rich queries
- Fast in-place updates
- Professional support by MongoDB

Where to Use MongoDB?

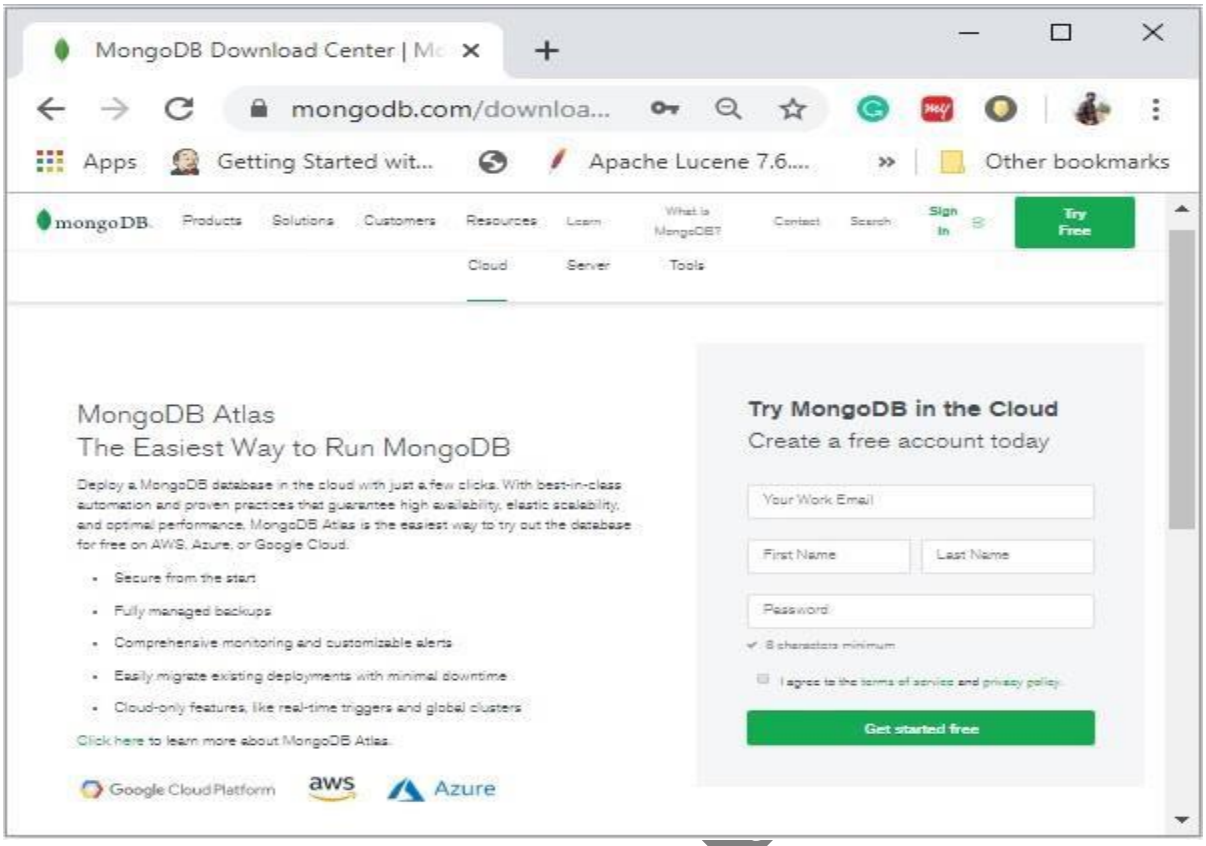
- Big Data
- Content Management and Delivery
- Mobile and Social Infrastructure
- User Data Management
- Data Hub

Let us now see how to install MongoDB on Windows.

Install MongoDB On Windows

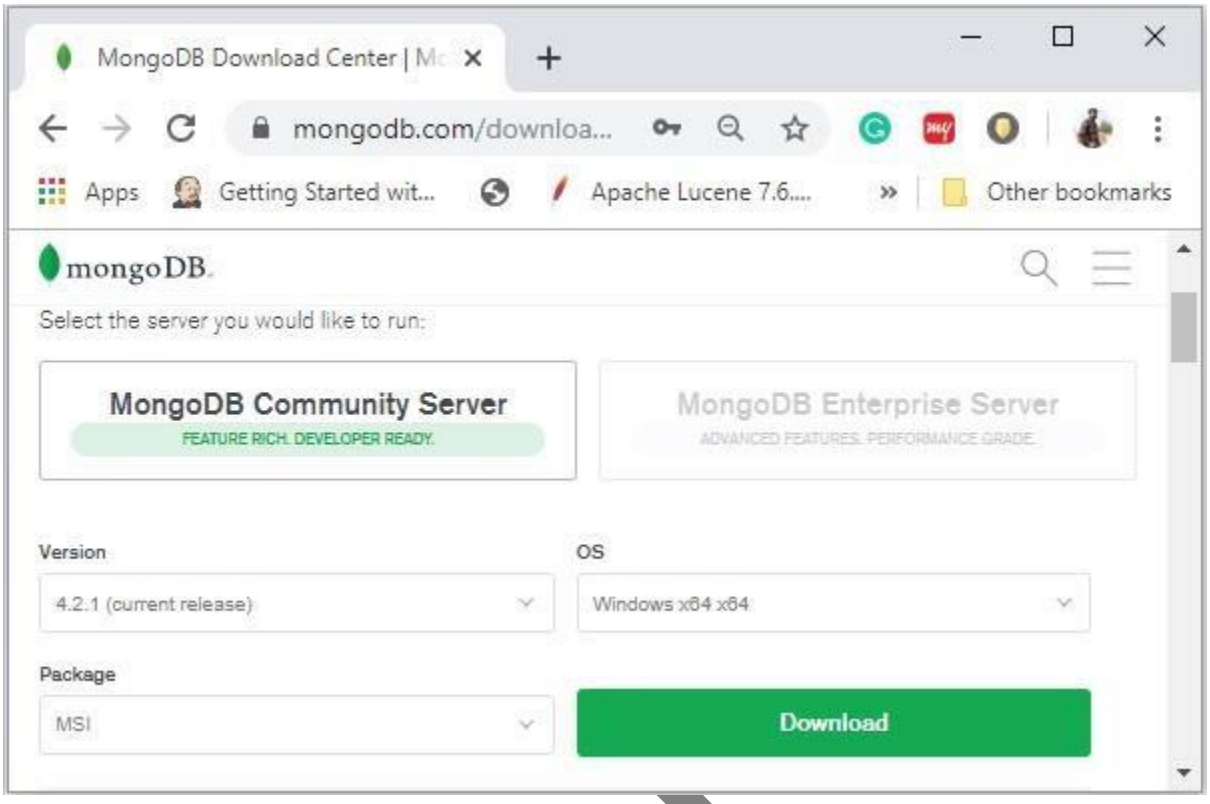
To install MongoDB on Windows, first download the latest release of MongoDB from https://www.mongodb.com/download-center

Esha Patel



Enter the required details, select the ***Server*** tab, in it you can choose the version of MongoDB, operating system and, packaging as:

Esha



Now install the downloaded file, by default, it will be installed in the folder **C:\Program Files**.

MongoDB requires a data folder to store its files. The default location for the MongoDB data directory is c:\data\db. So you need to create this folder using the Command Prompt. Execute the following command sequence.

```
C:\>md data
C:\md data\db
```

Then you need to specify set the **dbpath** to the created directory in **mongod.exe**. For the same, issue the following commands.

In the command prompt, navigate to the bin directory current in the MongoDB installation folder. Suppose my installation folder is **C:\Program Files\MongoDB**

```
C:\Users\XYZ>d:cd C:\Program Files\MongoDB\Server\4.2\bin
C:\Program Files\MongoDB\Server\4.2\bin>mongod.exe --dbpath "C:\data"
```

This will show **waiting for connections** message on the console output, which indicates that the mongod.exe process is running successfully.

Now to run the MongoDB, you need to open another command prompt and issue the following command.

Esha P

```
C:\Program Files\MongoDB\Server\4.2\bin>mongo.exe
MongoDB shell version v4.2.1
connecting to:
mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("4260beda-f662-4cbe-9bc7-5c1f2242663c") }
MongoDB server version: 4.2.1
>
```

This will show that MongoDB is installed and run successfully. Next time when you run MongoDB, you need to issue only commands.

```
C:\Program Files\MongoDB\Server\4.2\bin>mongod.exe --dbpath "C:\data"
C:\Program Files\MongoDB\Server\4.2\bin>mongo.exe
```

Start MongoDB

sudo service mongodb start

Stop MongoDB

sudo service mongodb stop

Restart MongoDB

sudo service mongodb restart

To use MongoDB run the following command.

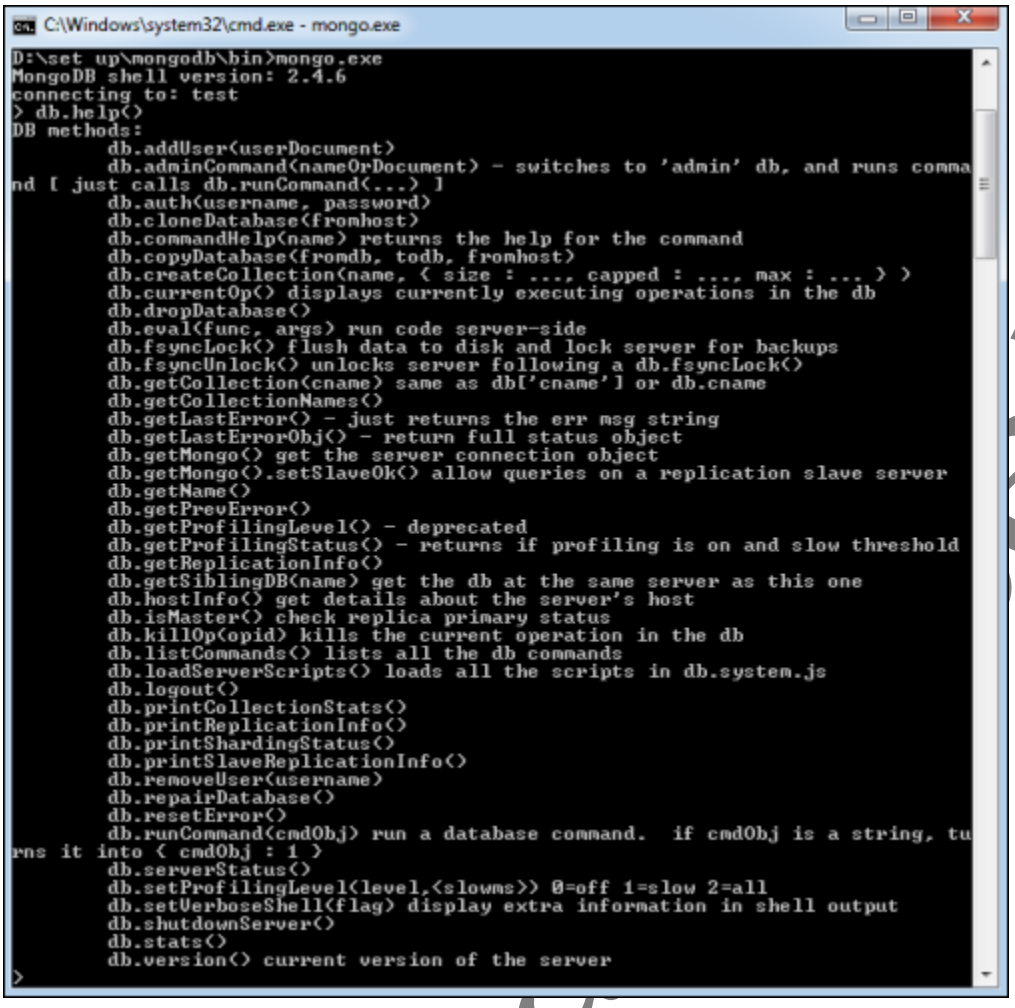
mongo

This will connect you to running MongoDB instance.

MongoDB Help

To get a list of commands, type **db.help()** in MongoDB client. This will give you a list of commands as shown in the following screenshot.

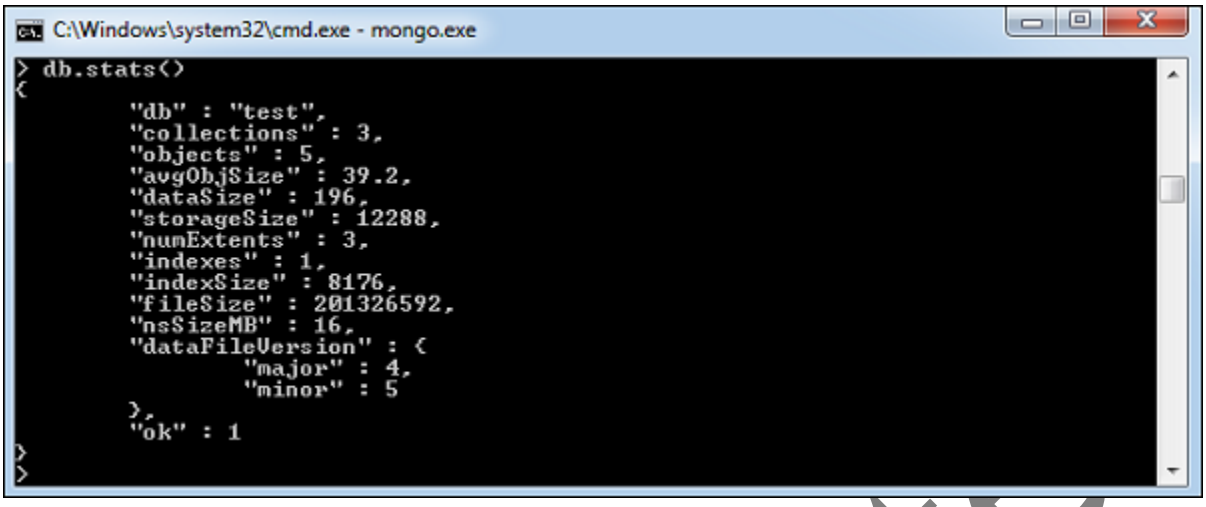
Esha Patel



MongoDB Statistics

To get stats about MongoDB server, type the command **db.stats()** in MongoDB client. This will show the database name, number of collection and documents in the database. Output of the command is shown in the following screenshot.

Esha Patel


C:\Windows\system32\cmd.exe - mongo.exe
> db.stats()
{
"db" : "test",
"collections" : 3,
"objects" : 5,
"avgObjSize" : 39.2,
"dataSize" : 196,
"storageSize" : 12288,
"numExtents" : 3,
"indexes" : 1,
"indexSize" : 8176,
"fileSize" : 201326592,
"nsSizeMB" : 16,
"dataFileVersion" : {
"major" : 4,
"minor" : 5
},
"ok" : 1
}
>

Esha Pate

Data in MongoDB has a flexible schema.documents in the same collection. They do not need to have the same set of fields or structure Common fields in a collection's documents may hold different types of data.

Data Model Design

MongoDB provides two types of data models: — Embedded data model and Normalized data model. Based on the requirement, you can use either of the models while preparing your document.

Embedded Data Model

In this model, you can have (embed) all the related data in a single document, it is also known as de-normalized data model.

For example, assume we are getting the details of employees in three different documents namely, Personal_details, Contact and, Address, you can embed all the three documents in a single one as shown below –

```
{
        _id: ,
        Emp_ID: "10025AE336"
        Personal_details:{
                First_Name: "Radhika",
                Last_Name: "Sharma",
                Date_Of_Birth: "1995-09-26"
        },
        Contact: {
                e-mail: "radhika_sharma.123@gmail.com",
                phone: "9848022338"
        },
        Address: {
                city: "Hyderabad",
                Area: "Madapur",
                State: "Telangana"
        }
}
```

Normalized Data Model

In this model, you can refer the sub documents in the original document, using references. For example, you can re-write the above document in the normalized model as:

Esha Patel

Employee:

```
{
        _id: <ObjectId101>,
        Emp_ID: "10025AE336"
}
```

Personal_details:

```
{
        _id: <ObjectId102>,
        empDocID: " ObjectId101",
        First_Name: "Radhika",
        Last_Name: "Sharma",
        Date_Of_Birth: "1995-09-26"
}
```

Contact:

```
{
        _id: <ObjectId103>,
        empDocID: " ObjectId101",
        e-mail: "radhika_sharma.123@gmail.com",
        phone: "9848022338"
}
```

Address:

```
{
        _id: <ObjectId104>,
        empDocID: " ObjectId101",
        city: "Hyderabad",
        Area: "Madapur",
        State: "Telangana"
}
```

Esha Patel

Considerations while designing Schema in MongoDB

- Design your schema according to user requirements.
- Combine objects into one document if you will use them together. Otherwise separate them (but make sure there should not be need of joins).

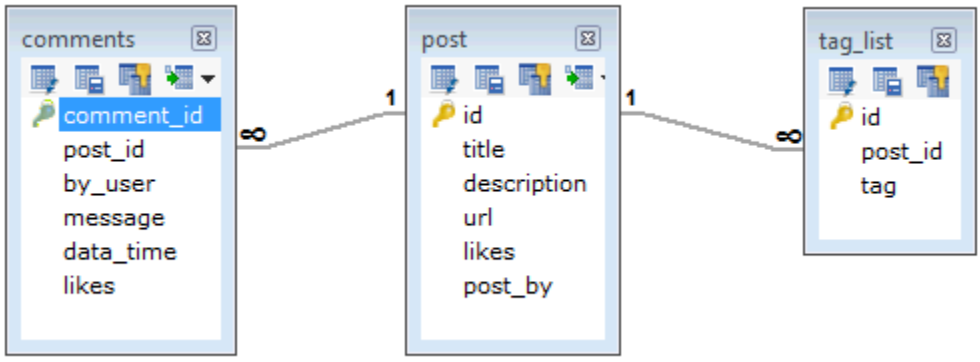
- Duplicate the data (but limited) because disk space is cheap as compare to compute time.
- Do joins while write, not on read.
- Optimize your schema for most frequent use cases.
- Do complex aggregation in the schema.

Example

Suppose a client needs a database design for his blog/website and see the differences between RDBMS and MongoDB schema design. Website has the following requirements.

- Every post has the unique title, description and url.
- Every post can have one or more tags.
- Every post has the name of its publisher and total number of likes.
- Every post has comments given by users along with their name, message, data-time and likes.
- On each post, there can be zero or more comments.

In RDBMS schema, design for above requirements will have minimum three tables.



While in MongoDB schema, design will have one collection post and the following structure –

```
{
  _id: POST_ID
  title: TITLE_OF_POST,
  description: POST_DESCRIPTION,
  by: POST_BY,
  url: URL_OF_POST,
  tags: [TAG1, TAG2, TAG3],
  likes: TOTAL_LIKES,
  comments: [
```

Esha Patel

```
    {
      user:'COMMENT_BY',
      message: TEXT,
      dateCreated: DATE_TIME,
      like: LIKES
    },
    {
      user:'COMMENT_BY',
      message: TEXT,
      dateCreated: DATE_TIME,
      like: LIKES
    }
  ]
}
```

So while showing the data, in RDBMS you need to join three tables and in MongoDB, data will be shown from one collection only.

1.1.2 Database creation and dropping database

Database creation

The use Command

MongoDB **use DATABASE_NAME** is used to create database or change database. The command will create a new database if it doesn't exist, otherwise it will return the existing database.

Syntax

Basic syntax of **use DATABASE** statement is as follows –

```
use DATABASE_NAME
```

Example

If you want to use a database with name **<mydb>**, then **use DATABASE** statement would be as follows –

```
>use mydb
switched to db mydb
```

To **check the currently selected database**, use the command **db**

Esha Patel

```
>db
mydb
```

To check your databases list, use the command **show dbs**.

```
>show dbs
local     40.00KiB
admin  40.00KiB
config  60.00KiB
```

Dropping database

The dropDatabase() Method

MongoDB **db.dropDatabase()** command is used to drop a existing database.

Syntax

Basic syntax of **dropDatabase()** command is as follows –

```
db. dropDatabase()
```

This will delete the selected database. If you have not selected any database, then it will delete default 'test' database.

Example

First, check the list of available databases by using the command, **show dbs**.

```
>show dbs
local      0.78125GB
mydb       0.23012GB
test       0.23012GB
>
```

If you want to delete new database **<mydb>**, then **dropDatabase()** command would be as follows –

```
>use mydb
switched to db mydb
>db.dropDatabase()
>{ "dropped" : "mydb", "ok" : 1 }
>
```

Now check list of databases.

```
>show dbs
```

Esha Patel

```
local      0.78125GB
test       0.23012GB
>
```

1.2 create and Drop collections

Create Collection

In MongoDB, **db.createCollection(name, options)** is used to create collection. But usually you don't need to create collection. MongoDB creates collection automatically when you insert some documents. It will be explained later. First see how to create collection:

Syntax

Basic syntax of **createCollection()** command is as follows −

```
db.createCollection(name, options)
```

In the command, **name** is name of collection to be created. **Options** is a document and is used to specify configuration of collection.

Parameter	Type	Description
Name	String	Name of the collection to be created
Options	Document	(Optional) Specify options about memory size and indexing

Options parameter is optional, so you need to specify only the name of the collection. Following is the list of options you can use −

Field	Type	Description
capped	Boolean	(Optional) If true, enables a capped collection. Capped collection is a fixed size collection that automatically overwrites its oldest entries when it reaches its maximum size. **If you specify true, you need to specify size parameter also.**

Esha Patel

autoIndexId	Boolean	(Optional) If true, automatically create index on _id field.s Default value is false.
size	number	(Optional) Specifies a maximum size in bytes for a capped collection. **If capped is true, then you need to specify this field also.**
max	number	(Optional) Specifies the maximum number of documents allowed in the capped collection.

While inserting the document, MongoDB first checks size field of capped collection, then it checks max field.

Examples

Basic syntax of **createCollection()** method without options is as follows −

```
>use test
switched to db test
>db.createCollection("mycollection")
{ "ok" : 1 }
>
```

To **check the created collection**, use the command **show collections.**

```
>show collections
Mycollection
```

How does MongoDB crate collection automatically

MongoDB creates collections automatically when you insert some documents. For example: Insert a document named seomount into a collection named SSSIT. The operation will create the collection if the collection does not currently exist.

```
>db.mysql.insert({"name" : "mysql"})
>show collections
  mysql
```

If you want to see the inserted document, use the find() command.

Syntax

```
db.collection_name.find()
```

Drop collections

In MongoDB, **db.collection.drop()** method is used to drop a collection from a database. It completely removes a collection from the database and does not leave any indexes associated with the dropped collections.

The **db.collection.drop()** method does not take any argument and produce an error when it is called with an argument. This method removes all the indexes associated with the dropped collection.

Syntax

Basic syntax of **drop()** command is as follows –

```
>db.COLLECTION_NAME.drop()
```

Example

First, check the available collections into your database **mydb**.

```
>use mydb
switched to db mydb
>show collections
mycollection
```

Now drop the collection with the name **mycollection**.

```
>db.mycollection.drop()
true
```

Again check the list of collections into database.

```
>show collections
```

drop() method will return true, if the selected collection is dropped successfully, otherwise it will return false.

1.3 CRUD operations (Insert, update, delete, find, Query and Projection operators)

Mongodb is a document-oriented database program widely classified as a NoSQL database program. In MongoDB, the **CRUD** operation refers to the creating, reading, updating, and deleting documents.

MongoDB is use for various things like building an application (including web and mobile), or analysis of data, or an administrator of a MongoDB database, in all these cases we need to interact with the MongoDB server to perform certain operations like entering new data into the application, updating data into the application, deleting data from the application, and reading the data of the application.

MongoDB provides a set of some basic but most essential operations that will help you to easily interact with the MongoDB server and these operations are known as **CRUD operations**.

Create Operations

The create or insert operations are used to insert or add new documents in the collection. If a collection does not exist, then it will create a new collection in the database. You can perform, create operations using following methods provided by the MongoDB:

The insert() Method

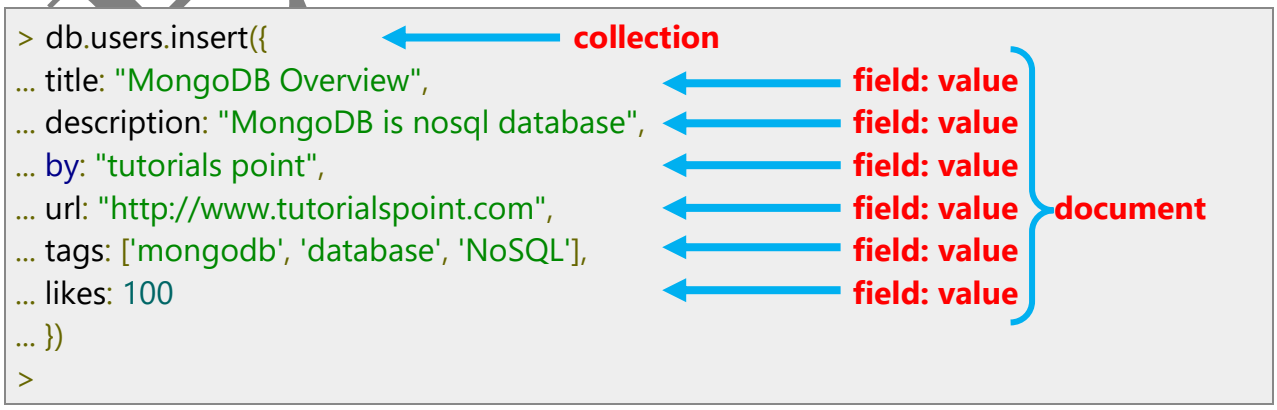
To insert data into MongoDB collection, you need to use MongoDB's **insert()** or **save()** method.

Syntax

The basic syntax of **insert()** command is as follows −

```
>db.COLLECTION_NAME.insert(document)
```

Example



Esha Patel

You can also pass an array of documents into the insert() method as shown below:.

```
> db.createCollection("post")
> db.post.insert([
{
        title: "MongoDB Overview",
        description: "MongoDB is no SQL database",
        by: "tutorials point",
        url: "http://www.tutorialspoint.com",
        tags: ["mongodb", "database", "NoSQL"],
        likes: 100
},
{
        title: "NoSQL Database",
        description: "NoSQL database doesn't have tables",
        by: "tutorials point",
        url: "http://www.tutorialspoint.com",
        tags: ["mongodb", "database", "NoSQL"],
        likes: 20,
        comments: [
        {
                user:"user1",
                message: "My first comment",
                dateCreated: new Date(2013,11,10,2,35),
                like: 0
        }
]
}
])
>
```

To insert the document you can use **db.post.save(document)** also. If you don't specify **_id** in the document then **save()** method will work same as **insert()** method. If you specify _id then it will replace whole data of document containing _id as specified in save() method.

The insertOne() method

If you need to insert only one document into a collection you can use this method.

Syntax

The basic syntax of insert() command is as follows −

```
>db.COLLECTION_NAME.insertOne(document)
```

Example

Following example creates a new collection named empDetails and inserts a document using the insertOne() method.

```
> db.createCollection("empDetails")
{ "ok" : 1 }
```

```
>db.empDetails.insertOne(
{
        First_Name: "Radhika",
        Last_Name: "Sharma",
        Date_Of_Birth: "1995-09-26",
        e_mail: "radhika123@gmail.com",
        phone: "9848022338"
})
```

Syntax

The basic syntax of insert() command is as follows −

```
>db.COLLECTION_NAME.insertMany(document)
```

The insertMany() method

You can insert multiple documents using the insertMany() method. To this method you need to pass an array of documents.

Example

Following example inserts three different documents into the empDetails collection using the insertMany() method.

```
> db.empDetails.insertMany(
        [
                {
                        First_Name: "Radhika",
                        Last_Name: "Sharma",
                        Date_Of_Birth: "1995-09-26",
                        e_mail: "radhika_sharma.123@gmail.com",
                        phone: "9000012345"
                },
                {
                        First_Name: "Rachel",
                        Last_Name: "Christopher",
```

```
                        Date_Of_Birth: "1990-02-16",
                        e_mail: "Rachel_Christopher.123@gmail.com",
                        phone: "9000054321"
                },
                {
                        First_Name: "Fathima",
                        Last_Name: "Sheik",
                        Date_Of_Birth: "1990-02-16",
                        e_mail: "Fathima_Sheik.123@gmail.com",
                        phone: "9000054321"
                }
        ]
)
```

Read Operations

The Read operations are used to retrieve documents from the collection, or in other words, read operations are used to query a collection for a document. You can perform read operation using the following method provided by the MongoDB:

The find() Method

To query data from MongoDB collection, you need to use MongoDB's **find()** method.

Syntax

The basic syntax of **find()** method is as follows –

```
>db.COLLECTION_NAME.find()
```

find() method will display all the documents in a non-structured way.

Example

Assume we have created a collection named mycol as –

```
> use sampleDB
switched to db sampleDB
> db.createCollection("mycol")
{ "ok" : 1 }
>
```

And inserted 3 documents in it using the insert() method as shown below –

```
> db.mycol.insert([
        {
                title: "MongoDB Overview",
```

```
                description: "MongoDB is no SQL database",
                by: "tutorials point",
                url: "http://www.tutorialspoint.com",
                tags: ["mongodb", "database", "NoSQL"],
                likes: 100
        },
        {
                title: "NoSQL Database",
                description: "NoSQL database doesn't have tables",
                by: "tutorials point",
                url: "http://www.tutorialspoint.com",
                tags: ["mongodb", "database", "NoSQL"],
                likes: 20,
                comments: [
                        {
                                user:"user1",
                                message: "My first comment",
                                dateCreated: new Date(2013,11,10,2,35),
                                like: 0
                        }
                ]
        }
])
```

Following method retrieves all the documents in the collection −

```
> db.mycol.find()
[
 {
  _id: ObjectId("64a11722e0de5ab72e1768e7"),
  title: 'MongoDB Overview',
  description: 'MongoDB is no SQL database',
  by: 'tutorials point',
  url: 'http://www.tutorialspoint.com',
  tags: [ 'mongodb', 'database', 'NoSQL' ],
  likes: 100
 },
 {
  _id: ObjectId("64a11722e0de5ab72e1768e8"),
  title: 'NoSQL Database',
  description: "NoSQL database doesn't have tables",
```

```
   by: 'tutorials point',
   url: 'http://www.tutorialspoint.com',
   tags: [ 'mongodb', 'database', 'NoSQL' ],
   likes: 20,
   comments: [
    {
      user: 'user1',
      message: 'My first comment',
      dateCreated: ISODate("2023-08-02T06:19:00.000Z"),
      like: 0
    }
  ]
 }
]
```

The pretty() Method

To display the results in a formatted way, you can use pretty() method.

Syntax

```
>db.COLLECTION_NAME.find().pretty()
```

The findOne() Method

Apart from the find() method, there is **findOne()** method, that returns only one document.

Syntax

```
>db.COLLECTIONNAME.findOne()
```

Example

Following example retrieves the document with title MongoDB Overview.

```
> db.mycol.findOne({title: "MongoDB Overview"})
{
        "_id" : ObjectId("5dd6542170fb13eec3963bf0"),
        "title" : "MongoDB Overview",
        "description" : "MongoDB is no SQL database",
        "by" : "tutorials point",
        "url" : "http://www.tutorialspoint.com",
        "tags" : ["mongodb","database","NoSQL"],
        "likes" : 100
}
```

Update Operations

MongoDB's **update()** and **save()** methods are used to update document into a collection. The update() method updates the values in the existing document while the save() method replaces the existing document with the document passed in save() method.

You can specify criteria, or filters, that identify the documents to update. These filters use the same syntax as read operations.

```
>db.users.deleteMany(          <-- collection
        {status: "reject"}     <-- delete filter
)
```

MongoDB Update() Method

The update() method updates the values in the existing document.

Syntax

The basic syntax of **update()** method is as follows –

```
>db.COLLECTION_NAME.update(SELECTION_CRITERIA, UPDATED_DATA)
```

Example

Consider the mycol collection has the following data.

```
{ "_id" : ObjectId(5983548781331adf45ec5), "title":"MongoDB Overview"}
{ "_id" : ObjectId(5983548781331adf45ec6), "title":"NoSQL Overview"}
{ "_id" : ObjectId(5983548781331adf45ec7), "title":"Tutorials Point Overview"}
```

Following example will set the new title 'New MongoDB Tutorial' of the documents whose title is 'MongoDB Overview'.

```
>db.mycol.update({'title':'MongoDB Overview'},{$set:{'title':'New MongoDB Tutorial'}})
>db.mycol.find()
{ "_id" : ObjectId(5983548781331adf45ec5), "title":"New MongoDB Tutorial"}
{ "_id" : ObjectId(5983548781331adf45ec6), "title":"NoSQL Overview"}
{ "_id" : ObjectId(5983548781331adf45ec7), "title":"Tutorials Point Overview"}
>
```

By default, MongoDB will update only a single document. To update multiple documents, you need to set a parameter 'multi' to true.

```
>db.mycol.update({'title':'MongoDB Overview'},
  {$set:{'title':'New MongoDB Tutorial'}},{multi:true})
```

Esha Patel

MongoDB Save() Method

This example is not run

The save() method replaces the existing document with the new document passed in the save() method.

Syntax

The basic syntax of MongoDB save() method is shown below –

```
>db.COLLECTION_NAME.save({_id:ObjectId(),NEW_DATA})
```

Example

Following example will replace the document with the _id '5983548781331adf45ec5'.

```
>db.mycol.save(
  {
    "_id" : ObjectId("507f191e810c19729de860ea"),
                    "title":"Tutorials Point New Topic",
    "by":"Tutorials Point"
  }
)
```

MongoDB findOneAndUpdate() method

The **findOneAndUpdate()** method updates the values in the existing document.

Syntax

The basic syntax of **findOneAndUpdate()** method is as follows –

```
>db.COLLECTION_NAME.findOneAndUpdate(SELECTIOIN_CRITERIA, UPDATED_DATA)
```

Example

Assume we have created a collection named empDetails and inserted three documents in it as shown below –

```
> db.empDetails.insertMany(
        [
                {
                        First_Name: "Radhika",
                        Last_Name: "Sharma",
                        Age: "26",
                        e_mail: "radhika_sharma.123@gmail.com",
```

Esha Patel

```
                    phone: "9000012345"
                },
                {
                    First_Name: "Rachel",
                    Last_Name: "Christopher",
                    Age: "27",
                    e_mail: "Rachel_Christopher.123@gmail.com",
                    phone: "9000054321"
                },
                {
                    First_Name: "Fathima",
                    Last_Name: "Sheik",
                    Age: "24",
                    e_mail: "Fathima_Sheik.123@gmail.com",
                    phone: "9000054321"
                }
        ]
)
```

Following example updates the age and email values of the document with name 'Radhika'.

```
> db.empDetails.findOneAndUpdate(
        {First_Name: 'Radhika'},
        { $set: { Age: '30',e_mail: 'radhika_newemail@gmail.com'}}
)
{
        "_id" : ObjectId("5dd6636870fb13eec3963bf5"),
        "First_Name" : "Radhika",
        "Last_Name" : "Sharma",
        "Age" : "30",
        "e_mail" : "radhika_newemail@gmail.com",
        "phone" : "9000012345"
}
>db.empDetails.find()  //To show the changes of collection data
```

MongoDB updateOne() method

This methods updates a single document which matches the given filter.

Syntax

The basic syntax of updateOne() method is as follows −

```
>db.COLLECTION_NAME.updateOne(<filter>, <update>)
```

Example

```
> db.empDetails.updateOne(
        {First_Name: 'Radhika'},
        { $set: { Age: '30',e_mail: 'radhika_newemail@gmail.com'}}
)
```

MongoDB updateMany() method

The updateMany() method updates all the documents that matches the given filter.

Syntax

The basic syntax of updateMany() method is as follows −

```
>db.COLLECTION_NAME.update(<filter>, <update>)
```

Example

```
> db.empDetails.updateMany(
        {Age:{ $gt: "25" }},
        { $set: { Age: '00'}}
)
```

You can see the updated values if you retrieve the contents of the document using the find method as shown below −

```
> db.empDetails.find()
[
 {
   _id:ObjectId(“64a053784f8fda2b751f5ccf”),
   FirstName:’Radhika’,
   LastName:’Sharma’
   Age:’00’,
   e_mail:’radhika123gmail.com’,
   phone:’9000012345’,
  },
  {
   _id:ObjectId(“”64a053784f8fda2b751f5cd0”),
   FirstName:’Rachel’,
   LastName:’Christopher’,
   Age:’00’
   e_mail:’Rachel_Christopher,123@gmail.com’,
```

```
   phone:'9000054321'
  },
  {
   _id:ObjectId("64a053784f8fda2b751f5cd0"),
   FirstName:'Fathima',
   LastName:'Sheik',
   Age:'24',
   e_mail:'Fathima_Sheik.123@gmail,com',
   phone:'9000054321'
} ]
```

Delete Operations

The delete operation are used to delete or remove the documents from a collection. You can perform delete operations using the following methods provided by the MongoDB:

You can specify criteria, or filters, that identify the documents to remove. These filters use the same syntax as read operations.

```
>db.users.deleteMany(          <-- collection
        {status: "reject"}     <-- delete filter
)
```

db.collection.deleteOne()

It is used to delete a single document from the collection that satisfy the given criteria.

or

It method removes only the first document matched by the query filter document.

Syntax

Basic syntax of **deleteOne()** method is as follows −

```
>db.COLLECTION_NAME.deleteOne (DELLETION_CRITTERIA)
```

Example

Consider the empd collection has the following data.

```
 [{
_id: ObjectId("649d8341682c1b6a14497710"),
First_Name: 'Rachel',
Last_Name: 'Christopher',
Date_Of_Birth: '1990-02-16',
```

```
e_mail: 'Rachel_Christopher.123@gmail.com',
phone: '9000054321'
},
{
_id: ObjectId("649d8341682c1b6a14497711"),
First_Name: 'Fathima',
Last_Name: 'Sheik',
Date_Of_Birth: '1990-02-16',
e_mail: 'Fathima_Sheik.123@gmail.com',
phone: '9000054321'
}
]
```

Following example will delete all the documents whose Last_Name is 'Sheik'.

```
>db.empd.deleteOne({ Last_Name: "Sheik"})
> db.emp.find()
[
  {
    _id: ObjectId("649d8341682c1b6a14497710"),
    First_Name: 'Rachel',
    Last_Name: 'Christopher',
    Date_Of_Birth: '1990-02-16',
    e_mail: 'Rachel_Christopher.123@gmail.com',
    phone: '9000054321'
  }
]
```

db.collection.deleteMany()

If you don't specify deletion criteria, then MongoDB will delete whole documents from the collection. **This is equivalent of SQL's truncate command.**

```
>db.empd.deleteMany({})
{ acknowledged: true, deletedCount: 1 }
> db.empd.find()
```

The remove() Method

MongoDB's **remove()** method is used to remove a document from the collection. remove() method accepts two parameters. One is deletion criteria and second is justOne flag.

- **deletion criteria** − (Optional) deletion criteria according to documents will be removed.
- **justOne** − (Optional) if set to true or 1, then remove only one document.

Syntax

Basic syntax of **remove()** method is as follows −

```
>db.COLLECTION_NAME.remove(DELLETION_CRITTERIA)
```

Example

```
{_id: ObjectId("649d8341682c1b6a14497710"), Last_Name:'Sheik'},
{_id:ObjectId("649d8341682c1b6a14497711"), Last_Name: 'Christopher'}
```

Following example will remove all the documents whose Last_Name is 'Sheik'.

```
>db.empd.remove({ Last_Name: "Sheik"})
> db.emp.find()
[
 {
   _id: ObjectId("649d8341682c1b6a14497710"),
   First_Name: 'Rachel',
   Last_Name: 'Christopher',
   Date_Of_Birth: '1990-02-16',
   e_mail: 'Rachel_Christopher.123@gmail.com',
   phone: '9000054321'
 }
]
```

Remove Only One

If there are multiple records and you want to delete only the first record, then set **justOne** parameter in **remove()** method.

```
>db.COLLECTION_NAME.remove(DELETION_CRITERIA,1)
```

Remove All Documents

If you don't specify deletion criteria, then MongoDB will delete whole documents from the collection. **This is equivalent of SQL's truncate command.**

```
>db.empd.remove({})
{acknowledged: true, deletedCount:2}
>
```

Query and Projection operators

MongoDB Query operators

There are many query operators that can be used to compare and reference document fields. The MongoDB query operator includes comparison, logical, element, evaluation, Geospatial, array, bitwise, and comment operators.

RDBMS Where Clause Equivalents in MongoDB(Comparison Operators)

To query the document on the basis of some condition, you can use following operations.

Operation	Syntax	Example	RDBMS Equivalent
Equality	{<key>:{$eg;<value>}}	db.mycol.find({"by":"tutorials point"}).pretty()	where by = 'tutorials point'
Less Than	{<key>:{$lt:<value>}}	db.mycol.find({"likes":{$lt:50}}).pretty()	where likes < 50
Less Than Equals	{<key>:{$lte:<value>}}	db.mycol.find({"likes":{$lte:50}}).pretty()	where likes <= 50
Greater Than	{<key>:{$gt:<value>}}	db.mycol.find({"likes":{$gt:50}}).pretty()	where likes > 50
Greater Than Equals	{<key>:{$gte:<value>}}	db.mycol.find({"likes":{$gte:50}}).pretty()	where likes >= 50

Esha Patel

Not Equals	{<key>:{$ne:<value>}}	db.mycol.find({"likes":{$ne:50}}).pretty()	where likes != 50
Values in an array	{<key>:{$in:[<value1>, <value2>,.....<valueN>]}}	db.mycol.find({"name":{$in:["Raj", "Ram", "Raghu"]}}).pretty()	Where name matches any of the value in :["Raj", "Ram", "Raghu"]
Values not in an array	{<key>:{$nin:<value>}}	db.mycol.find({"name":{$nin:["Ramu", "Raghav"]}}).pretty()	Where name values is not in the array :["Ramu", "Raghav"] or, doesn't exist at all

Mongodb logical Operator

AND in MongoDB

Syntax

To query documents based on the AND condition, you need to use $and keyword. Following is the basic syntax of AND –

```
>db.mycol.find({ $and: [ {<key1>:<value1>}, { <key2>:<value2>} ] })
```

Example

Following example will show all the tutorials written by 'tutorials point' and whose title is 'MongoDB Overview'.

```
> db.mycol.find({$and:[{"by":"tutorials point"},{"title": "MongoDB Overview"}]}).pretty()
{
        "_id" : ObjectId("5dd4e2cc0821d3b44607534c"),
        "title" : "MongoDB Overview",
        "description" : "MongoDB is no SQL database",
        "by" : "tutorials point",
        "url" : "http://www.tutorialspoint.com",
        "tags" : [
                "mongodb",
                "database",
```

Esha Patel

```
                    "NoSQL"
            ],
            "likes" : 100
}
>
```

For the above given example, equivalent where clause will be **' where by = 'tutorials point' AND title = 'MongoDB Overview' '**. You can pass any number of key, value pairs in find clause.

OR in MongoDB

Syntax

To query documents based on the OR condition, you need to use **$or** keyword. Following is the basic syntax of **OR** −

```
>db.mycol.find(
  {
    $or: [
      {key1: value1}, {key2:value2}
    ]
  }
).pretty()
```

Example

Following example will show all the tutorials written by 'tutorials point' or whose title is 'MongoDB Overview'.

```
>db.mycol.find({$or:[{"by":"tutorials point"},{"title": "MongoDB Overview"}]}).pretty()
{
  "_id": ObjectId(7df78ad8902c),
  "title": "MongoDB Overview",
  "description": "MongoDB is no sql database",
  "by": "tutorials point",
  "url": "http://www.tutorialspoint.com",
  "tags": ["mongodb", "database", "NoSQL"],
  "likes": "100"
}
>
```

Using AND and OR Together

Example

The following example will show the documents that have likes greater than 10 and whose title is either 'MongoDB Overview' or by is 'tutorials point'. Equivalent SQL where clause is **'where likes>10 AND (by = 'tutorials point' OR title = 'MongoDB Overview')'**

```
>db.mycol.find({"likes": {$gt:10}, $or: [{"by": "tutorials point"},
   {"title": "MongoDB Overview"}]}).pretty()
{
   "_id": ObjectId(7df78ad8902c),
   "title": "MongoDB Overview",
   "description": "MongoDB is no sql database",
   "by": "tutorials point",
   "url": "http://www.tutorialspoint.com",
   "tags": ["mongodb", "database", "NoSQL"],
   "likes": "100"
}
>
```

NOR in MongoDB

Syntax

To query documents based on the NOT condition, you need to use $not keyword. Following is the basic syntax of **NOT** –

```
>db.COLLECTION_NAME.find(
        {
                $not: [
                        {key1: value1}, {key2:value2}
                ]
        }
)
```

Example

Assume we have inserted 3 documents in the collection **empDetails** as shown below –

```
db.empDetails.insertMany(
        [
                {
                        First_Name: "Radhika",
                        Last_Name: "Sharma",
```

```
                        Age: "26",
                        e_mail: "radhika_sharma.123@gmail.com",
                        phone: "9000012345"
                },
                {
                        First_Name: "Rachel",
                        Last_Name: "Christopher",
                        Age: "27",
                        e_mail: "Rachel_Christopher.123@gmail.com",
                        phone: "9000054321"
                },
                {
                        First_Name: "Fathima",
                        Last_Name: "Sheik",
                        Age: "24",
                        e_mail: "Fathima_Sheik.123@gmail.com",
                        phone: "9000054321"
                }
        ]
)
```

Following example will retrieve the document(s) whose first name is not "Radhika" and last name is not "Christopher"

```
> db.empDetails.find(
        {
                $nor:[
                        40
                        {"First_Name": "Radhika"},
                        {"Last_Name": "Christopher"}
                ]
        }
).pretty()
{
        "_id" : ObjectId("5dd631f270fb13eec3963bef"),
        "First_Name" : "Fathima",
        "Last_Name" : "Sheik",
        "Age" : "24",
        "e_mail" : "Fathima_Sheik.123@gmail.com",
        "phone" : "9000054321"
```

```
}
```

NOT in MongoDB

Syntax

To query documents based on the NOT condition, you need to use $not keyword following is the basic syntax of **NOT** –

```
>db.COLLECTION_NAME.find(
        {
                $NOT: [
                        {key1: value1}, {key2:value2}
                ]
        }
).pretty()
```

Example

Following example will retrieve the document(s) whose age is not greater than 25

```
> db.empDetails.find( { "Age": { $not: { $gt: "25" } } } )
{
        "_id" : ObjectId("5dd6636870fb13eec3963bf7"),
        "First_Name" : "Fathima",
        "Last_Name" : "Sheik",
        "Age" : "24",
        "e_mail" : "Fathima_Sheik.123@gmail.com",
        "phone" : "9000054321"
}
```

Mongodb Element Operator

$exists

The exists operator matches the documents that contain the field when Boolean is true. It also matches the document where the field value is null.

Syntax: { field: { $exists: <boolean> } }
Example: db.books.find ({ qty: { $exists: **true**, $nin: [5, 15] } })

$type

The type operator chooses documents where the value of the field is an instance of the specified BSON type.

Syntax: { field: { $type: <BSON type> } }
Example: db.books.find ({ "bookid" : { $type : 2 } });

Mongodb Evaluation Operator

$expr

The expr operator allows the use of aggregation expressions within the query language.

Syntax: { $expr: { <expression> } }
Example: db.store.find({ $expr: {$gt: ["$product" , "price"] } })

$jsonSchema

It matches the documents that satisfy the specified JSON Schema.

Syntax: { $jsonSchema: <JSON **schema** object> }

$mod

The mod operator selects the document where the value of a field is divided by a divisor has the specified remainder.

Syntax: { field: { $mod: [divisor, remainder] } }
Example: db.books.find ({ qty: { $mod: [200, 0] } })

$regex

It provides regular expression abilities for pattern matching strings in queries. The MongoDB uses regular expressions that are compatible with Perl.

Syntax: { <field>: /pattern/<options> }
Example: db.books.find({ price: { $regex: /789$/ } })

$text

The $text operator searches a text on the content of the field, indexed with a text index.

Syntax: {
$text:

```
        {
          $search: <string>,
          $language: <string>,
          $caseSensitive: <Boolean>,
        $diacriticSensitive: <Boolean>
           }
        }
```

Example: db.books.find({ $text: { $search: "Othelo" } })

$where

The "where" operator is used for passing either a string containing a JavaScript expression or a full JavaScript function to the query system.

Example: db.books.find ({ $**where**: **function**() {
return (hex_md5(this.**name**)== "9b53e667f30cd329dca1ec9e6a8")
}});

MongoDB Geospatial Operator

$geoIntersects

It selects only those documents whose geospatial data intersects with the given GeoJSON object.

Syntax: {
<location field>: {
$geoIntersects: {
$geometry: {
type: "<object type>" ,
coordinates: [<coordinates>]
}
}
}
}

Example: db.places.find({
loc: {
$geoIntersects: {
$geometry: {
type: "Triangle" ,

```
        coordinates: [
          [ [ 0, 0 ], [ 3, 6 ], [ 6, 1 ] ]
         ]
       }
     }
   }
 }
```

$geoWithin

The geoWithin operator chooses the document with geospatial data that exists entirely within a specified shape.

```
Syntax: {
<location field>: {
$geoWithin: {
  $geometry: {
     type: <"Triangle" or "Rectangle"> ,
    coordinates: [ <coordinates> ]
   }
  }
}
```

$near

The near operator defines a point for which a geospatial query returns the documents from close to far.

```
Syntax: {
<location field>: {
   $near: {
    $geometry: {
       type: "Point" ,
       coordinates: [ <longitude> , <latitude> ]
     },
     $maxDistance: <distance in meters>,
     $minDistance: <distance in meters>
   }
  }
```

Example:

```
db.places.find({
 location:
  {$near: {
    $geometry: { type: "Point",  coordinates: [ -73.9667, 40.78 ] },
      $minDistance: 1000,
      $maxDistance: 5000
     }
   }
 }
```

$nearSphere

The nearsphere operator specifies a point for which the geospatial query returns the document from nearest to farthest.

Syntax: {
$nearSphere: [<x>, <y>],
$minDistance: <distance in radians>,
$maxDistance: <distance in radians>
}

Example:

```
db.legacyPlaces.find(
{ location : { $nearSphere : [ -73.9667, 40.78 ], $maxDistance: 0.10 } }
}
```

$all

It chooses the document where the value of a field is an array that contains all the specified elements.

Syntax: { <field>: { $all: [<value1> , <value2> ...] } }
Example: db.books.find({ tags: { $all: ["Java", "MongoDB", "RDBMS"] } })

$elementMatch

The operator relates documents that contain an array field with at least one element that matches with all the given query criteria.

Syntax: { <field>: { $elemMatch: { <query1>, <query2>, ... } } }
Example: db.books.find(
{ results: { $elemMatch: { $gte: 500, $lt: 400 } } }
}

$size

It selects any array with the number of the element specified by the argument.

Syntax: db.collection.find({ field: { $**size**: 2 } });

MongoDB Bitwise Operator

$bitsAllClear

It matches the documents where all the bit positions given by the query are clear infield.

Syntax: { <field>: { $bitsAllClear: <**numeric** bitmask> } }
Example: db.inventory.find({ a: { $bitsAllClear: [1, 5] } })

$bitsAllSet

The bitallset operator matches the documents where all the bit positions given by the query are set in the field.

Syntax: { <field>: { $bitsAllSet: <**numeric** bitmask> } }
Example: db.inventory.find({ a: { $bitsAllClear: [1, 5] } })

$bitsAnyClear

The bitAnyClear operator matches the document where any bit of positions given by the query is clear in the field.

Syntax: { <field>: { $bitsAnyClear: <**numeric** bitmask> } }
Example: db.inventory.find({ a: { $bitsAnyClear: [5, 10] } })

$bitsAnySet

It matches the document where any of the bit positions given by the query are set in the field.

Syntax: { <field>: { $bitsAnySet: <**numeric** bitmask> } }
Example: db.inventory.find({ a: { $bitsAnySet: [1, 5] } })

MongoDB Comments Operator

$comment

The $comment operator associates a comment to any expression taking a query predicate.

Syntax: db.inventory.find({ <query>, $comment: <comment> })
Example: db.inventory.find({
x: { $mod: [1, 0] },
$comment: "Find Odd values."
}

MongoDB Projection Operator

$

The $ operator limits the contents of an array from the query results to contain only the first element matching the query document.

Syntax: db.books.find({ <array>: <value> ... },
{ "<array>.$": 1 })
db.books.find({ <array.field>: <value> ...},
{ "<array>.$": 1 })

$elemMatch

The content of the array field made limited using this operator from the query result to contain only the first element matching the element $elemMatch condition.

Syntax: db.library.find({ bookcode: "63109" },
{ students: { $elemMatch: { roll: 102 } } })

$meta

The meta operator returns the result for each matching document where the metadata associated with the query.

Syntax: { $meta: <metaDataKeyword> }
Example: db.books.find(
<query>,
{ score: { $meta: "textScore" } }

$slice

It controls the number of values in an array that a query returns.

Syntax: db.books.find({ field: value }, { array: {$slice: count } });
Example: db.books.find({}, { comments: { $slice: [200, 100] } })

1.4 Operators (Projection, update, limit (), sort ()) and Aggregation commands

MongoDB operators just like operators in other programming languages, MongoDB operators are symbols that tell the compiler or interpreter to perform the specific relational, mathematical, or logical operations and produce the final result.

MongoDB Projection Operator

$

The $ operator limits the contents of an array from the query results to contain only the first element matching the query document.

Syntax: db.books.find({ <array>: <value> ... },
{ "<array>.$": 1 })
db.books.find({ <array.field>: <value> ...},
{ "<array>.$": 1 })

$elemMatch

The content of the array field made limited using this operator from the query result to contain only the first element matching the element $elemMatch condition.

Syntax: db.library.find({ bookcode: "63109" },
{ students: { $elemMatch: { roll: 102 } } })

$meta

The meta operator returns the result for each matching document where the metadata associated with the query.

```
Syntax: { $meta: <metaDataKeyword> }
Example: db.books.find(
<query>,
{ score: { $meta: "textScore" } }
```

$slice

It controls the number of values in an array that a query returns.

```
Syntax: db.books.find( { field: value }, { array: {$slice: count } } );
Example: db.books.find( {}, { comments: { $slice: [ 200, 100 ] } } )
```

MongoDB Update Operator

The following modifiers are available to update operations. For example - in db.collection.update() and db.collection.findAndModify().

Defines the operator expression in the document of the form:

```
Syntax: <operator1>: { <field1>: <value1>, ... },
        <operator2>: { <field2>: <value2>, ... },
        }
```

Field operator

$currentDate

It updates the elements of a field to the current date, either as a Date or a timestamp. The default data type of this operator is the date.

```
Syntax: { $currentDate: { <field1>: <typeSpecification1>, ... } }
Example: db.books.insertOne(
 { _id: 1, status: "a", lastModified: purchaseDate("2013-10-02T01:11:18.965Z") }
)
```

$inc

It increases a filed by the specified value.

```
Syntax: { $inc: { <field1>: <amount1>, <field2>: <amount2>, ... } }
Example: {
 _id: 000438,
 sku: "MongoDB",
quantity: 1,
metrics: {
 orders: 2,
 ratings: 3.5
   }
  }
```

$min

It changes the value of the field to a specified value if the specified value is less than the current value of the filed.

```
Syntax: { $min: { <field1>: <value1>, ... } }
Example: { _id: 0021, highprice: 800, lowprice: 200 }
db.books.update( { _id: 0021 }, { $min: { highprice: 500 } } )
```

$max

It changes the value of the field to a specified value if the specified value is greater than the current value of the filed.

```
Syntax: { $max: { <field1>: <value1>, ... } }
Example: { _id: 0021, highprice: 800, lowprice: 200 }
db.books.update( { _id: 0021 }, { $max: { highprice: 950 } } )
```

$mul

It multiplies the value of a field by a number.

```
Syntax: { $mul: { <field1>: <number1>, ... } }
Example: db.books.update(
 { _id: 1 },
{ $mul: { price: NumberDecimal("180.25"), qty: 2 } }
```

```
)
```

$rename

The rename operator changes the name of a field.

```
Syntax: {$rename: { <field1>: <newName1>, <field2>: <newName2>, ... } }
Example: db.books.updateMany( {}, { $rename: { "nmae": "name" } } )
```

$set

The set operator changes the value of a field with the specified value.

```
Syntax: { $set: { <field1>: <value1>, ... } }
Example:{
_id: 100,
sku: "abc123",
quantity: 50,
instock: true,
reorder: false,
details: { model: "14Q2", make: "xyz" },
tags: [ "technical", "non technical" ],
ratings: [ { by: "ijk", rating: 4 } ]
}
```

$setOnInsert

If the upsert is set to true, then it results in an insert of a document, then setOnInsert operator assigns the specified values to the field in the document.

```
Syntax: db.collection.update(
            <query>,
            { $setOnInsert: { <field1>: <value1>, ... } },
             { upsert: true }
})
```

$unset

It removes a specified field.

```
Syntax: { $unset: { <field1>: "", ... } }
```

```
Example:  db.products.update(
          { sku: "unknown" },
          { $unset: { quantity: "", instock: "" } })
```

Array Operators

$

We can update an element in an array without explicitly specifying the position of the element.

```
Syntax: { "<array>.$" : value }
Example: db.collection.update(
          { <array>: value ... },
          { <update operator>: { "<array>.$" : value } }
```

$[]

The positional operator indicates that the update operator should change all the elements in the given array field.

```
Syntax: { <update operator>: { "<array>.$[]" : value } }
Example: db.collection.updateMany(
          { <query conditions> },
          { <update operator>: { "<array>.$[]" : value } }
```

$[<identifier>]

It is called a filtered positional operator that identifies the array elements.

```
Syntax: { <update operator>: { "<array>.$[<identifier>]" : value } },
          { arrayFilters: [ { <identifier>: <condition> } ] }
Example: db.collection.updateMany( { <query conditions> },
          { <update operator>: { "<array>.$[<identifier>]" : value } },
          { arrayFilters: [ { <identifier>: <condition> } ] } )
```

$addToSet

It adds an element to an array unless the element is already present, in which case this operator does nothing to that array.

Syntax: { $addToSet: { <field1>: <value1>, ... } }
Example: db.books.**update**(
{ _id: 1 },
{ $addToSet: { tags: "MongoDB" } }

$pop

We can remove the first or last element of an array using the pop operator. We need to pass the value of pop as -1 to remove the first element of an array and 1 to remove the last element in an array.

Syntax: { $pop: { <field>: <-1 | 1>, ... } }
Example: db.books.**update**({ _id: 1 }, { $pop: { mongoDB: -1 } })

$pull

Using a pull operator, we can remove all the instances of a value in an array that matches the specified condition.

Syntax: { $pull: { <field1>: <value|condition>, <field2>: <value|condition>, ... } }
Example:
db.books.**update**({ }, { $pull: { Development: { $in:["Java", "RDBMS"] }, Tech: "Cybersecurity" } },
{ multi: **true** }
}

$push

It appends a specified value to an array.

Syntax: { $push: { <field1>: <value1>, ... } }
Example: db.students.**update**({ _id: 9 }, { $push: { scores: 91 } })

$pullAll

We can remove all instances of the specified value from existing array using the pullAll operator. It removes element that match the listed value.

Syntax: { $pullAll: { <field1>: [<value1>, <value2> ...], ... } }
Example: db.survey.**update**({ _id: 1 }, { $pullAll: { scores: [0, 5] } })

Modifiers

$each

It is used with the $addToSet operator and the $push operator. It is used with the addToSet operator to add multiple values to an array if the value does not exist in the field.

Syntax: { $addToSet: { <field>: { $each: [<value1>, <value2> ...] } } }

It is used with the push operator to append multiple values to an array.

Syntax: { $push: { <field>: { $each: [<value1>, <value2> ...] } } }
Example:
db.students.**update**({ **name**: "Akki" }, { $push: { scores: { $each: [90, 92, 85] } } })

$position

It specifies the location where the push operator inserts elements inside an array.

Syntax: {
$push:{
<field>: {
$each: [<value1>, <value2>, ...],
$position: <num>
}
}
Example: db.students.**update**(
{ _id: 1 },
{
$push: {
scores: {
$each: [50, 60, 70],
$position: 0
}
}
})

$slice

This modifier is used to limit the number of array elements during the push operation.

```
Syntax: {
$push: {
<field>: {
$each: [ <value1>, <value2>, ... ],
 $slice: <num>
  }
}
Example: db.students.update(
              { _id: 1 }, {
             $push: {
             scores: {
             $each: [ 80, 78, 86 ],
              $slice: -5
                }
              }
            }
          )
```

$sort

The sort modifier arranges the values of an array during the push operation.

```
Syntax: {
$push: {
<field>: {
$each: [ <value1>, <value2>, ... ],
 $sort: <sort specification>
  }
}
Example: db.students.update(
 { _id: 1 }, {
$push: {
quizzes: {
$each: [ { id: 3, score: 8 }, { id: 4, score: 7 }, { id: 5, score: 6 } ],
 $sort: { score: 1 }
             }
```

```
        }
      }
  }
)
```

Bitwise Operator

$bit

The bit operator updates a field using a bitwise operation. It supports bitwise AND, bitwise OR, and bitwise XOR operations.

Syntax: { $**bit**: { <field>: { <and|or|xor>: <**int**> } } }
Example: db.books.**update**({ _id: 1 }, { $**bit**: { expdata: { and: price(100) } } })

MongoDB Limit() Method

In MongoDB, limit() method is used to limit the fields of document that you want to show. Sometimes, you have a lot of fields in collection of your database and have to retrieve only 1 or 2. In such case, limit() method is used. To limit the records in MongoDB, you need to use **limit()** method. The method accepts one number type argument, which is the number of documents that you want to be displayed.

The MongoDB limit() method is used with find() method.

Syntax

The basic syntax of **limit()** method is as follows −

```
>db.COLLECTION_NAME.find().limit(NUMBER)
```

Example

Consider the collection myycol has the following data.

```
{_id : ObjectId("507f191e810c19729de860e1"), title: "MongoDB Overview"},
{_id : ObjectId("507f191e810c19729de860e2"), title: "NoSQL Overview"},
{_id : ObjectId("507f191e810c19729de860e3"), title: "Tutorials Point Overview"}
```

Following example will display only two documents while querying the document.

```
>db.mycol.find({},{"title":1,_id:0}).limit(2)
{"title":"MongoDB Overview"}
{"title":"NoSQL Overview"}
>
```

If you don't specify the number argument in **limit()** method then it will display all documents from the collection.

MongoDB Skip() Method

Apart from limit() method, there is one more method **skip()** which also accepts number type argument and is used to skip the number of documents.

Syntax

The basic syntax of **skip()** method is as follows −

```
>db.COLLECTION_NAME.find().limit(NUMBER).skip(NUMBER)
```

Example

Following example will display only the second document.

```
>db.mycol.find({},{"title":1,_id:0}).limit(1).skip(1)
{"title":"NoSQL Overview"}
>
```

Note: The default value in **skip()** method is 0.

MongoDB sort() method

In MongoDB, sort() method is used to sort the documents in the collection. The method accepts a document containing a list of fields along with their sorting order.

The sorting order is specified as 1 or -1.

- 1 is used for ascending order sorting.
- -1 is used for descending order sorting.

Syntax

The basic syntax of **sort()** method is as follows −

```
>db.COLLECTION_NAME.find().sort({KEY:1})
```

Example

Consider the collection myycol has the following data.

```
{_id : ObjectId("507f191e810c19729de860e1"), title: "MongoDB Overview"}
{_id : ObjectId("507f191e810c19729de860e2"), title: "NoSQL Overview"}
{_id : ObjectId("507f191e810c19729de860e3"), title: "Tutorials Point Overview"}
```

Following example will display the documents sorted by title in the descending order.

```
>db.mycol.find({},{"title":1,_id:0}).sort({"title":-1})
{"title":"Tutorials Point Overview"}
{"title":"NoSQL Overview"}
{"title":"MongoDB Overview"}
>
```

Note: if you don't specify the sorting preference, then sort() method will display the documents in ascending order.

MongoDB Aggregation commands

Aggregations operations process data records and return computed results. Aggregation operations group values from multiple documents together, and can perform a variety of operations on the grouped data to return a single result. In SQL count(*) and with group by is an equivalent of MongoDB aggregation.

The aggregate() Method

For the aggregation in MongoDB, you should use **aggregate()** method.

Syntax

Basic syntax of **aggregate()** method is as follows −

```
>db.COLLECTION_NAME.aggregate(AGGREGATE_OPERATION)
```

Example

In the collection you have the following data −

```
{
   _id: ObjectId(7df78ad8902c)
   title: 'MongoDB Overview',
   description: 'MongoDB is no sql database',
   by_user: 'tutorials point',
   url: 'http://www.tutorialspoint.com',
   tags: ['mongodb', 'database', 'NoSQL'],
   likes: 100
},
{
   _id: ObjectId(7df78ad8902d)
   title: 'NoSQL Overview',
   description: 'No sql database is very fast',
   by_user: 'tutorials point',
   url: 'http://www.tutorialspoint.com',
   tags: ['mongodb', 'database', 'NoSQL'],
   likes: 10
},
{
   _id: ObjectId(7df78ad8902e)
   title: 'Neo4j Overview',
   description: 'Neo4j is no sql database',
   by_user: 'Neo4j',
   url: 'http://www.neo4j.com',
   tags: ['neo4j', 'database', 'NoSQL'],
   likes: 750
},
```

Now from the above collection, if you want to display a list stating how many tutorials are written by each user, then you will use the following **aggregate()** method −

```
> db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$sum : 1}}}])
{ "_id" : "tutorials point", "num_tutorial" : 2 }
{ "_id" : "Neo4j", "num_tutorial" : 1 }
>
```

Sql equivalent query for the above use case will be **select by_user, count(*) from mycol group by by_user**.

In the above example, we have grouped documents by field **by_user** and on each occurrence of by user previous value of sum is incremented. Following is a list of available aggregation expressions.

Expression	Description	Example
$sum	Sums up the defined value from all documents in the collection.	db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$sum : "$likes"}}}])
$avg	Calculates the average of all given values from all documents in the collection.	db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$avg : "$likes"}}}])
$min	Gets the minimum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$min : "$likes"}}}])
$max	Gets the maximum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$max : "$likes"}}}])
$push	Inserts the value to an array in the resulting document.	db.mycol.aggregate([{$group : {_id : "$by_user", url : {$push: "$url"}}}])
$addToSet	Inserts the value to an array in the resulting document but does not create duplicates.	db.mycol.aggregate([{$group : {_id : "$by_user", url : {$addToSet : "$url"}}}])
$first	Gets the first document from the source documents according to the grouping. Typically this makes only sense together with some previously applied “$sort”-stage.	db.mycol.aggregate([{$group : {_id : "$by_user", first_url : {$first : "$url"}}}])
$last	Gets the last document from the source documents according to the grouping. Typically this makes only sense together with some previously applied “$sort”-stage.	db.mycol.aggregate([{$group : {_id : "$by_user", last_url : {$last : "$url"}}}])

Esha Patel

Pipleline Concept

In UNIX command, shell pipeline means the possibility to execute an operation on some input and use the output as the input for the next command and so on. MongoDB also supports same concept in aggregation framework. There is a set of possible stages and each of those is taken as a set of documents as an input and produces a resulting set of documents (or the final resulting JSON document at the end of the pipeline). This can then in turn be used for the next stage and so on.

Following are the possible stages in aggregation framework −

- **$project** − Used to select some specific fields from a collection.
- **$match** − This is a filtering operation and thus this can reduce the amount of documents that are given as input to the next stage.
- **$group** − This does the actual aggregation as discussed above.
- **$sort** − Sorts the documents.
- **$skip** − With this, it is possible to skip forward in the list of documents for a given amount of documents.
- **$limit** − This limits the amount of documents to look at, by the given number starting from the current positions.
- **$unwind** − This is used to unwind document that are using arrays. When using an array, the data is kind of pre-joined and this operation will be undone with this to have individual documents again. Thus with this stage we will increase the amount of documents for the next stage.

Esha Patel